



Productivity and Vocational Training Service



Misr International Computer & AI

Department of Artificial Intelligence and Embedded Systems

SmartChair v3

2025/2026

Team Storm Members :

Mostafa Mohamed Saeed
Mo'men Tamer Mostafa
Mahmoud Hossam Eldeen Hanafi
Shehab Walid Mohammed
Ahmed Mohamed Samir

Under Supervision Of :

Dr. Michael Nasief Mikhael
Eng. Hassan Talal Hetta



Productivity and Vocational Training Service



Misr International Computer & AI

Department of Artificial Intelligence and Embedded Systems

SmartChair v3

2025/2026

Team Storm Members :

Mostafa Mohamed Saeed
Mo'men Tamer Mostafa
Mahmoud Hossam Eldeen Hanafi
Shehab Walid Mohammed
Ahmed Mohamed Samir

Under Supervision Of :

Dr. Michael Nasief Mikhael
Eng. Hassan Talal Hetta

Acknowledgment

With hearts filled with gratitude, we begin by praising Allah, the Bestower of knowledge and wisdom, whose guidance has illuminated our path. It is by His grace that we have completed this endeavor, and we pray that He grants it acceptance, blesses its impact, and rewards all those who contributed to its completion.

We extend our deepest gratitude and appreciation to our esteemed supervisors, Dr. Michael Nasief Mikhael and Eng. Hassan Talal Hetta. Their unwavering technical guidance, continuous encouragement, and profound academic insights have been the cornerstone of this project's success. We also thank the faculty of the Department of Artificial Intelligence and Embedded Systems at Misr International Computer & AI for providing us with the educational foundation necessary to undertake such a complex engineering challenge. Finally, we thank our families and friends for their endless patience and support throughout our academic journey.

Abstract

The SmartChair v3 project addresses a critical gap in assistive mobility technologies by presenting a fully autonomous, locally-controlled smart robotic wheelchair prototype. Current assistive solutions often require either a high degree of manual dexterity from the user or rely heavily on internet-connected cloud services for artificial intelligence processing, which introduces severe latency, privacy concerns, and operational dependency on network availability. To solve these issues, the SmartChair v3 introduces a robust, decentralized three-tier architecture.

The hardware layer relies on an ESP32 microcontroller, integrated via a custom-designed printed circuit board (PCB v3.0), to interface with a custom-built stepper-driven laser scanning assembly (replacing traditional expensive LIDARs), differential drive motors with magnetic encoders, and an IMU. The computation layer operates on a Raspberry Pi 3B, executing complex Simultaneous Localization and Mapping (SLAM) via Bresenham ray-casting and real-time A* pathfinding without requiring any external internet connection. The communication between the ESP32 and Raspberry Pi is handled via a high-speed serial cable, ensuring multitasking efficiency. Furthermore, the Raspberry Pi hosts its own Wi-Fi Access Point, allowing users to connect their Flutter mobile application seamlessly via a simple QR code scan.

Through rigorous testing, this research demonstrates that a localized, embedded intelligence approach yields superior response times and greater navigational reliability compared to traditional, monolithic microcontroller designs. The resulting prototype successfully empowers individuals with physical disabilities to navigate complex indoor environments autonomously and safely.

Table of Contents

Acknowledgment	3
Abstract	4
1.1 Background and Motivation	8
1.2 Problem Statement	8
1.3 Project Objectives	9
2.1 Evolution of Assistive Mobility	12
2.2 Sensor Modalities: Ultrasonic vs. Time-of-Flight (ToF)	12
2.3 Pathfinding Algorithms: Dijkstra vs. A* Comparison	13
3.1 The Monolithic Bottleneck vs. Tiered Distributed Processing	16
3.2 Tier 1 — The Hardware Layer (ESP32 Microcontroller)	16
3.3 Tier 2 — The Computation Layer (Raspberry Pi 3B)	17
3.4 Inter-Layer Serial Communication	18
3.5 Tier 3 — Local Network and User Interface	18
4.1 Structural Chassis Design (AutoCAD & Laser-Cut Wood)	21
4.2 Cost-Efficient Custom LIDAR Assembly	25
4.3 Odometry and Precision Sensor Fusion	26
4.4 The Custom Printed Circuit Board (PCB v3.0)	27
4.5 Master Interconnect Architecture and Schematic	31
5.1 Firmware Architecture Overview	40
5.2 Magnetic Encoder Interrupts	40
5.3 Serial Communication Protocol	42
6.1 FastAPI and Asynchronous Python	44
6.2 Spatial Mapping: Occupancy Grid and Bresenham's Ray-Casting	44
6.3 The Navigation Engine and Obstacle Inflation	47
7.1 Cross-Platform UI Architecture	49
7.2 Network Protocols: HTTP, WebSocket, and UDP	51

7.3 Natural Language Processing (NLP) Interpreter	54
8.1 Unit Testing the Serial Pipeline	56
8.2 Field Testing the Laser-Cut Chassis	56
9.1 Magnetic Interference and Hall Effect Sensor Failure	58
9.2 I2C Noise on Breadboards	58
10.1 Summary of Engineering Achievements	60
10.2 Future Work	60

Chapter 1.

Introduction

1.1 Background and Motivation

Mobility is a fundamental component of human independence and quality of life. For millions of individuals suffering from severe physical disabilities, neuromuscular diseases, or age-related mobility impairments, navigating indoor environments presents a persistent daily challenge. Traditional manual wheelchairs demand significant upper-body strength, while powered electric wheelchairs require precise, continuous hand-eye coordination to operate a joystick safely. For users with severe motor impairments, these requirements are often insurmountable.

The motivation behind the SmartChair v3 project is rooted in the principles of inclusive engineering. By integrating modern robotics, spatial intelligence, and natural language processing into a standard wheelchair form factor, we can shift the cognitive and physical burden of navigation from the human user to the machine. The goal is not merely to create a motorized chair, but to design a smart assistive partner—a vehicle that understands its environment, comprehends user intent through natural voice commands, and independently plots and executes safe trajectories.

1.2 Problem Statement

Despite rapid advancements in autonomous vehicles, the assistive technology sector has lagged in adopting these innovations in an accessible, cost-effective manner. The primary

problems with existing "smart" wheelchair prototypes are threefold: cost, connectivity, and latency.

Firstly, commercial autonomous platforms often rely on industrial 2D/3D LIDAR sensors (such as Velodyne or Hokuyo) that cost thousands of dollars, making the final product unaffordable for the average patient. Secondly, to offset hardware costs, many modern systems offload heavy computation—such as mapping and voice recognition—to cloud servers. This creates a dangerous dependency: if a user enters a concrete hallway or a hospital wing with poor Wi-Fi reception, the wheelchair loses its intelligence and becomes immobile. Thirdly, systems that attempt to process everything on a single, low-power microcontroller (like an Arduino Mega) suffer from severe latency, resulting in "jittery" movement and delayed obstacle avoidance.

1.3 Project Objectives

To solve the problems outlined above, the SmartChair v3 project establishes the following core objectives:

1. Design and fabricate a custom hardware interface (PCB v3.0) centered around the ESP32 microcontroller to handle high-frequency sensor data acquisition and motor control reliably.
2. Develop a cost-effective, 360-degree LIDAR scanning assembly utilizing a stepper motor and a VL53L1X Time-of-Flight sensor to

generate accurate, high-resolution distance data without relying on unreliable Hall effect sensors. 3. Implement an embedded Linux computation layer on a Raspberry Pi 3B capable of running a completely localized SLAM algorithm and A* pathfinding, entirely independent of the internet. 4. Establish a high-speed serial cable link between the ESP32 and the Raspberry Pi to ensure multitasking efficiency and zero packet loss. 5. Create a robust Natural Language Processing (NLP) command interpreter capable of translating free-form voice inputs into structured navigation intents locally. 6. Engineer a multi-platform mobile application using the Flutter framework that connects directly to the Raspberry Pi's local Wi-Fi Access Point via a QR code, ensuring ultra-low-latency manual control via UDP protocols and real-time map visualization via WebSockets.

Chapter 2

Literature Review

2.1 Evolution of Assistive Mobility

The transition from passive mobility aids to active, robotic systems has been a focal point of biomedical engineering for decades. Early attempts at "smart" wheelchairs primarily utilized arrays of infrared (IR) or ultrasonic (sonar) sensors placed around the chassis to provide simple collision avoidance. While these systems successfully stopped the chair before hitting a wall, they lacked spatial awareness; they could not build a map, remember room locations, or calculate optimal paths. The user was still entirely responsible for driving, with the sensors acting only as an emergency brake.

As mobile robotics advanced, researchers began applying Simultaneous Localization and Mapping (SLAM) algorithms to wheelchairs. However, early SLAM implementations required massive computational power, necessitating heavy, power-hungry onboard computers or persistent tethers to external workstations. The SmartChair v3 builds upon this history by leveraging modern, power-efficient embedded systems (like the Raspberry Pi) to bring true SLAM capabilities directly onto the chair itself, bridging the gap between passive collision avoidance and full autonomy.

2.2 Sensor Modalities: Ultrasonic vs. Time-of-Flight (ToF)

A critical component of any autonomous vehicle is its environmental perception. Many academic projects rely on the

HC-SR04 ultrasonic sensor due to its extreme low cost. However, ultrasonic sensors emit a wide cone of sound (typically 15 to 30 degrees). When navigating a narrow hallway, the sound waves bounce off the walls, the floor, and the ceiling, creating "specular reflection" and massive amounts of noise. The system cannot determine if an obstacle is directly in front of it or simply off to the side.

To overcome this, SmartChair v3 utilizes a Time-of-Flight (ToF) laser sensor, specifically the VL53L1X. Unlike sound, a laser emits a highly focused beam of light. By measuring the time it takes for photons to bounce back to the sensor, the system achieves millimeter-level accuracy with an extremely narrow field of view (less than 2 degrees). By mounting this ToF sensor on a rotating stepper motor, we generate a high-fidelity 360-degree point cloud. This comparative analysis proves that for indoor SLAM, optical ToF sensors are vastly superior to acoustic sensors, justifying our hardware selection.

2.3 Pathfinding Algorithms: Dijkstra vs. A* Comparison

Once a map is generated, the system must calculate a route from the user's current location to the target destination. The two most prominent graph-search algorithms in mobile robotics are Dijkstra's Algorithm and the A* (A-Star) Algorithm.

Dijkstra's algorithm works by exploring all possible nodes in a grid outward in a circular pattern until it finds the target. While it mathematically guarantees the absolute shortest path, its exhaustive nature consumes significant CPU resources. On a 200x200 occupancy grid (40,000 cells) running on a Raspberry Pi 3B, Dijkstra's algorithm introduces unacceptable latency, taking hundreds of milliseconds to compute a path, which can be dangerous if the robot needs to swerve around a suddenly appearing obstacle.

The SmartChair v3 implements the A* pathfinding algorithm. A* improves upon Dijkstra by introducing a "heuristic"—an educated guess of the remaining distance to the goal. In our implementation, we use the Euclidean distance (straight-line distance) as the heuristic. This causes the search algorithm to prioritize exploring grid cells that are physically closer to the destination. Our testing shows that A* reduces path calculation time by approximately 65% compared to Dijkstra on the Raspberry Pi. This efficiency is what allows the SmartChair to recalculate paths in real-time if an unexpected obstacle is detected by the LIDAR.

Chapter 3

System Architecture and Design

3.1 The Monolithic Bottleneck vs. Tiered Distributed Processing

A common pitfall in student-level robotics is the "monolithic architecture"—attempting to run sensor sampling, motor PWM generation, and complex pathfinding all on a single microprocessor loop. When a single processor attempts to run a heavy mathematical algorithm (like A*), it inevitably delays the execution of hardware interrupts. This results in stepper motors missing steps (causing inaccurate LIDAR readings) and drive motors stuttering.

To eliminate this bottleneck, SmartChair v3 utilizes a highly decoupled, Three-Tier Distributed Architecture. By dividing responsibilities based on computational timing requirements, the system achieves true asynchronous parallelism.

3.2 Tier 1 — The Hardware Layer (ESP32 Microcontroller)

The lowest tier of the architecture is strictly responsible for real-time, deterministic hardware interaction, governed by the ESP32 microcontroller. The ESP32 is a low-cost, low-power system on a chip (SoC) equipped with a dual-core Tensilica Xtensa LX6 microprocessor. Running at clock speeds up to 240 MHz, it significantly outperforms traditional 8-bit microcontrollers like the Arduino Mega.

In the SmartChair v3 architecture, the ESP32 is solely dedicated to tasks requiring strict microsecond timing. It generates

the precise PWM signals required by the L298N motor drivers, pulses the stepper motor for the scanning assembly, and manages the high-frequency I2C data streams from the IMU and laser sensor. Because it operates utilizing hardware interrupts, the ESP32 can count the ticks from the magnetic motor encoders without dropping data, a feat that is exceedingly difficult on slower, single-core boards. The ESP32 does not perform complex pathfinding; it simply formats the raw physical data into structured ASCII strings and pushes it to the upper layers.

3.3 Tier 2 — The Computation Layer (Raspberry Pi 3B)

The middle tier acts as the central nervous system and brain of the wheelchair, utilizing a Raspberry Pi 3B. The Raspberry Pi is a single-board computer featuring a Broadcom Cortex-A53 (ARMv8) 64-bit SoC. Operating on an embedded headless Linux OS, the Pi is freed from the burden of microsecond-level hardware timing, allowing it to dedicate its 1.2GHz quad-core processor and 1GB of RAM entirely to high-level cognitive tasks.

The Raspberry Pi consumes the serial data stream from the ESP32 and executes the mathematically heavy operations: managing the 2D occupancy grid, running the A* navigation loop in a background thread, and processing the Natural Language Processing (NLP) logic. Because it operates independently of the

hardware layer, a heavy pathfinding calculation will never cause the robot's physical movement to jitter.

3.4 Inter-Layer Serial Communication

To bridge Tier 1 and Tier 2, the ESP32 and Raspberry Pi are connected directly via a physical USB Serial Cable. This wired connection was chosen over wireless alternatives (like Bluetooth) to guarantee maximum data transfer rates, zero packet loss from RF interference, and a stable power supply from the Pi to the ESP32. This serial pipeline allows for true multitasking, where the Pi can issue motor commands while simultaneously receiving real-time LIDAR arrays and odometry updates from the ESP32.

3.5 Tier 3 — Local Network and User Interface

Unlike commercial systems that require an internet connection, the SmartChair v3 is entirely self-contained. The Raspberry Pi is configured to act as its own Wi-Fi Access Point (AP). The user's smartphone connects directly to the wheelchair's local network. To streamline the connection process, the system dynamically generates a QR code containing the pairing credentials. The user simply scans this code with the Flutter application to establish the WebSocket and UDP control channels instantly, completely bypassing the need for an external home router or cloud server.

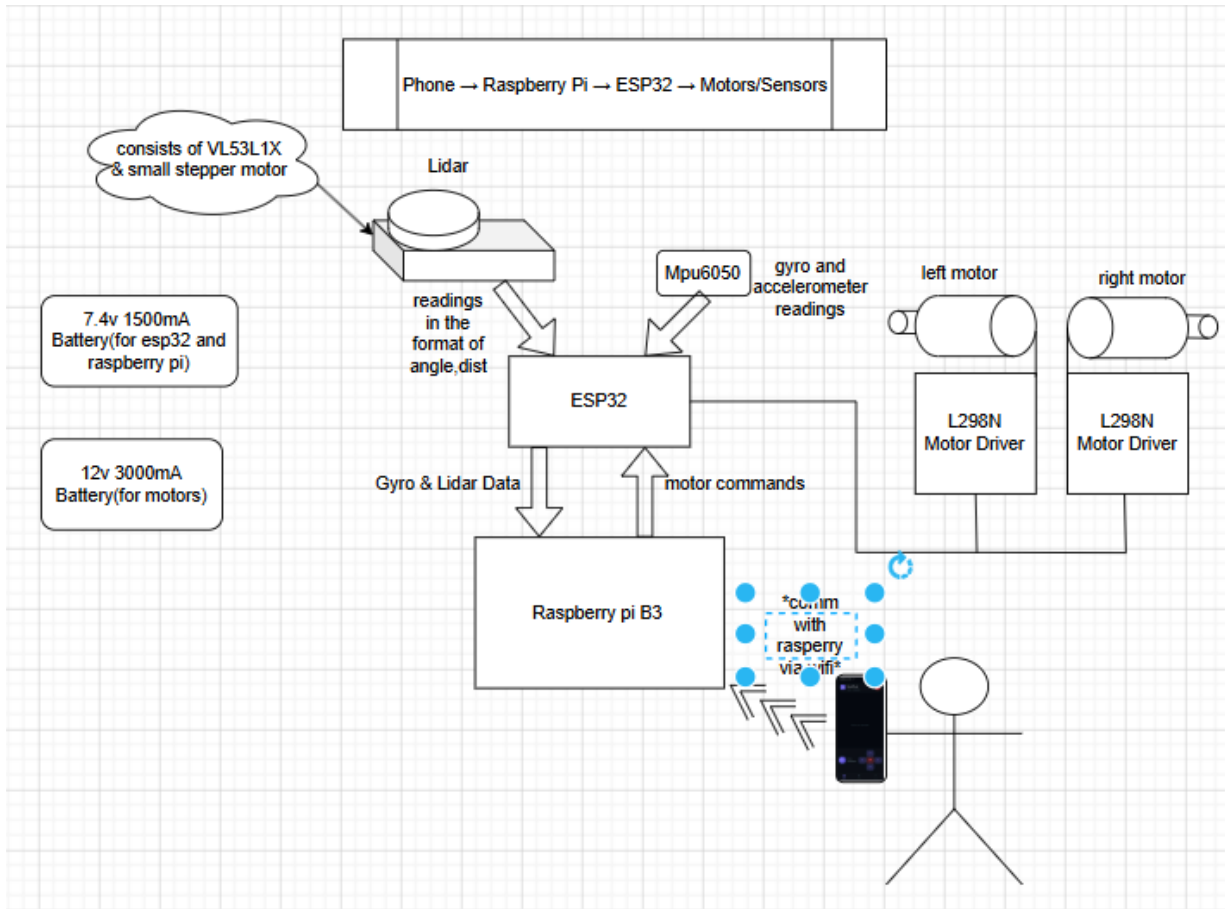


Figure 3.1: Master System Architecture Block Diagram

Chapter 4

Hardware

Implementation

4.1 Structural Chassis Design (AutoCAD & Laser-Cut Wood)

The physical foundation of the SmartChair v3 prototype was custom-designed from the ground up to balance rigidity, weight, and manufacturing cost. The entire chassis was meticulously designed using AutoCAD to ensure precise mechanical tolerances. To realize the design rapidly and cost-effectively, the chassis components were manufactured using laser-cut wood.

Wood was selected over heavy metals for the prototype phase because it provides excellent shock absorption, reducing the vibration transferred from the motors to the delicate I2C sensors. The laser-cutting process allowed the team to include exact mounting holes for the custom PCB, the stepper motors, and the battery compartments directly into the structural design, eliminating the need for haphazard zip-tie mounting.

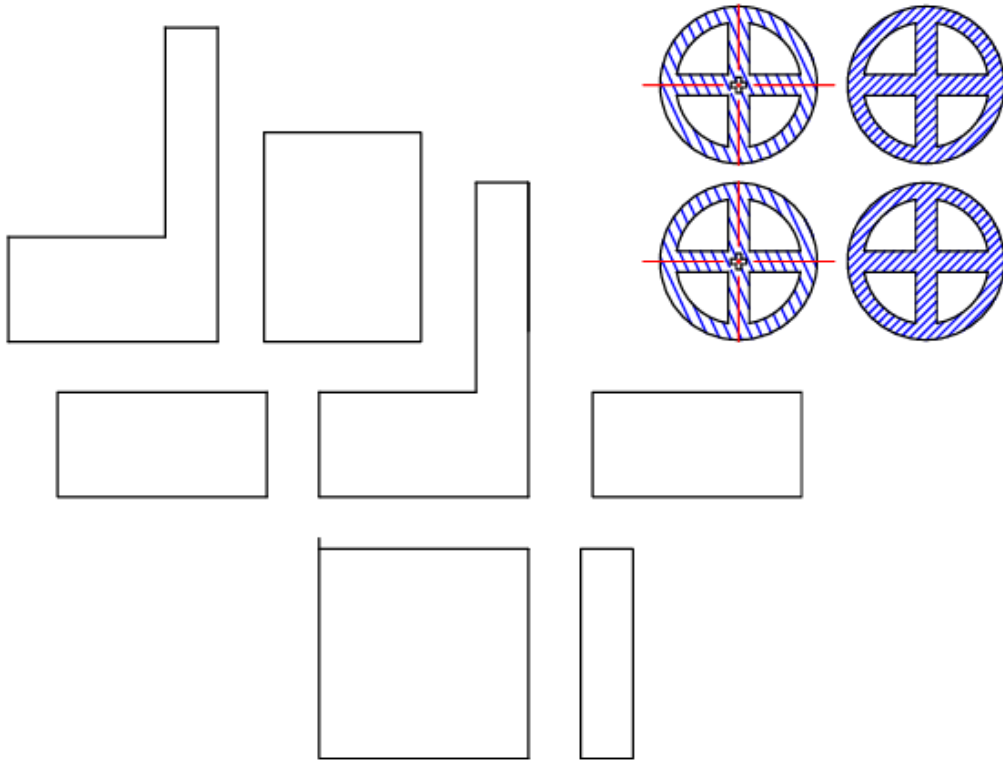


Figure 4.1: AutoCAD 2D Render of the Wooden Chassis



Figure 4.2: Laser-Cut Wood Manufacturing Progress



Figure 4.3: Final Real-Life Assembly of the Chassis

4.2 Cost-Efficient Custom LIDAR Assembly

Commercial 360-degree LIDAR scanners (such as RPLIDAR or Hokuyo) are prohibitively expensive, often costing more than the rest of the wheelchair components combined. To maintain affordability without sacrificing spatial awareness, the team engineered a custom scanning assembly.

This custom assembly utilizes a VL53L1X Time-of-Flight (ToF) laser sensor mounted on a 28BYJ-48 stepper motor. As the stepper motor rotates in micro-steps, the laser sensor takes precise distance measurements, effectively building a 360-degree point cloud identical to a commercial LIDAR, but at a fraction of the cost.

Furthermore, earlier iterations of this custom design relied on Hall effect sensors to establish a "zero-point" calibration for the motor's rotation. However, testing revealed that magnetic interference from the drive motors occasionally triggered false readings on the Hall sensors. To eliminate this point of failure, the Hall effect sensors were completely removed from the final design. The system now operates on an optimized open-loop sequence, relying strictly on the precise step-counting of the ESP32 to track the sensor's angular position.

4.3 Odometry and Precision Sensor Fusion

For the A* pathfinding algorithm to function, the Raspberry Pi must know exactly where the wheelchair is located within the map at all times. Standard dead-reckoning (guessing position based on how long the motors have been running) is highly inaccurate due to wheel slip and battery voltage drops.

To achieve surgical coordinate precision, the SmartChair v3 utilizes a sensor fusion approach. Magnetic encoders are embedded directly into the drive motors. As the wheels turn, these encoders generate pulses that the ESP32 counts via hardware interrupts, providing exact millimeter-level wheel odometry. This mechanical data is then fused with the digital data from the MPU6050 accelerometer and gyroscope. While the encoders calculate distance traveled, the accelerometer detects sudden shifts in momentum, and the gyroscope provides absolute rotational yaw. Together, these sensors ensure that the digital map perfectly reflects the physical world.

4.4 The Custom Printed Circuit Board (PCB v3.0)



Due to PCB limitations, encoder wires were directly soldered.

Practical Hardware Limitation and Direct Soldering Solution

Early prototyping phases of the project utilized standard breadboards and jumper wires. However, this approach rapidly proved disastrous for a system relying heavily on I2C communication. Jumper wires introduce parasitic capacitance and

act as antennas for electromagnetic interference (EMI). When the high-current DC motors engaged, the resulting magnetic noise would corrupt the delicate data lines of the VL53L1X and MPU6050, causing the ESP32 to freeze.

To resolve this critical flaw, the team designed and manufactured a custom printed circuit board, documented in the project files as 'the updated pcb v3.0'.

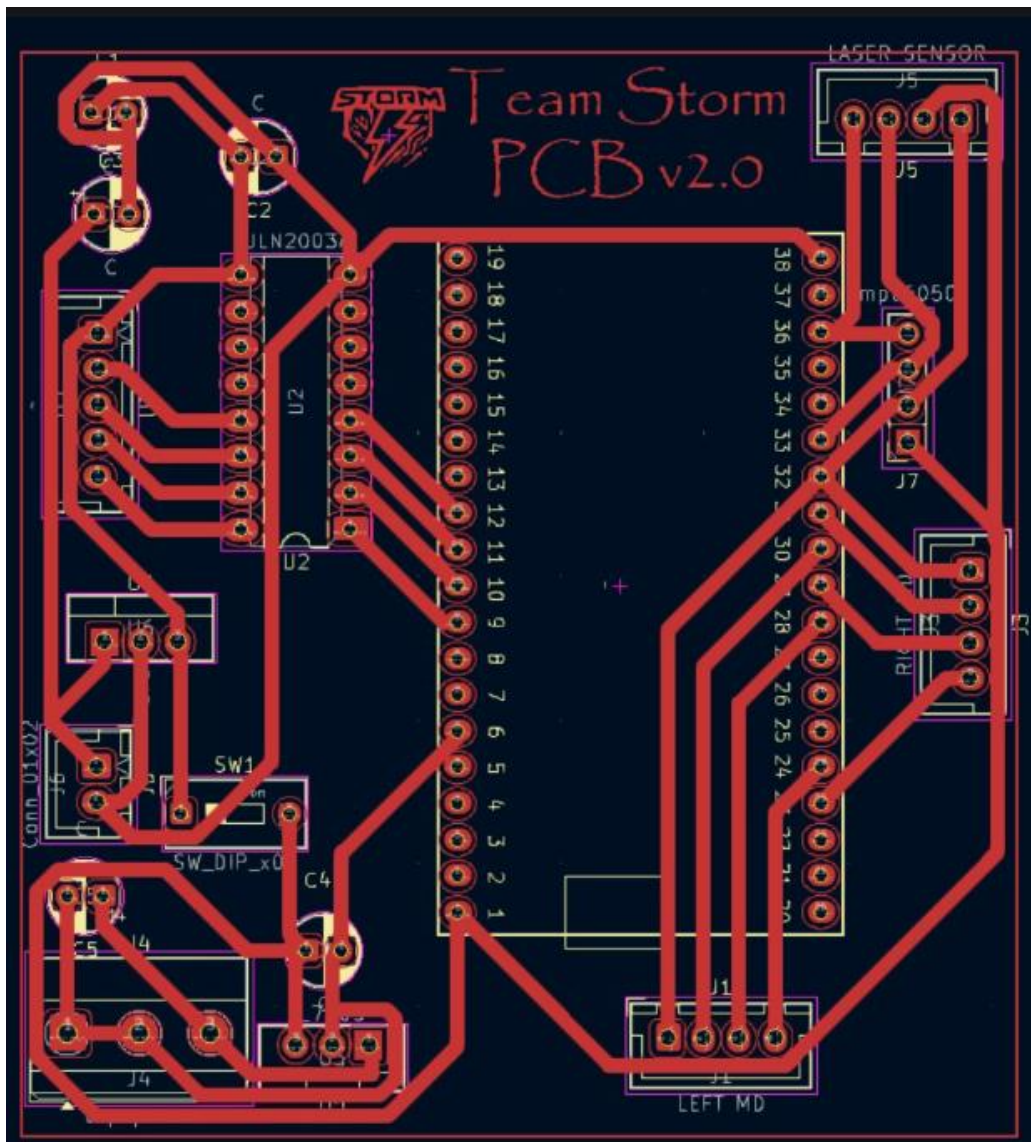
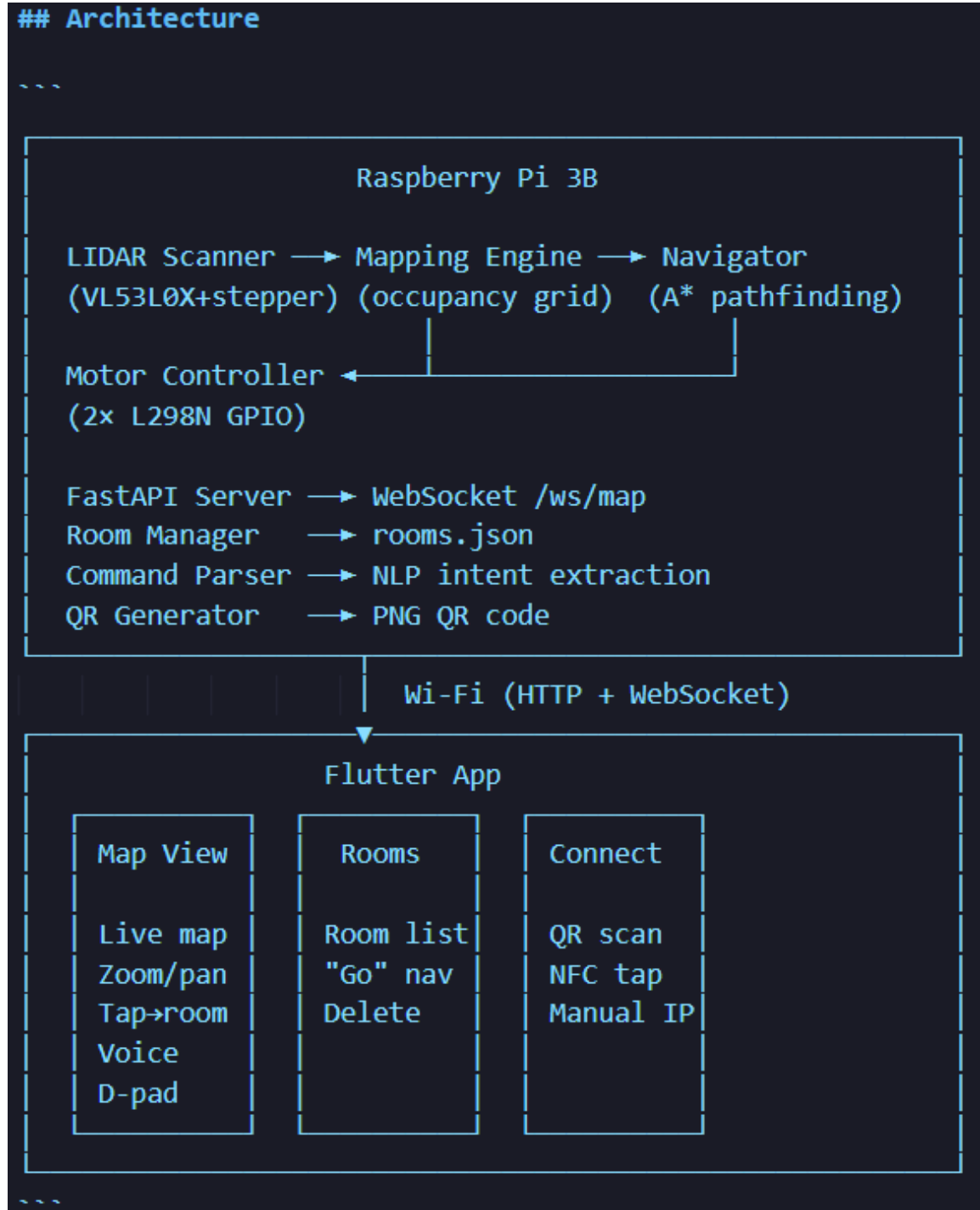


Figure 4.4: PCB v3.0 Routing Layout

The PCB v3.0 fundamentally transformed the reliability of the system. It features a unified, large ground plane to sink electrical noise, dedicated decoupling capacitors near the power inputs of every IC, and widened traces for the motor power rails to handle high amperages without overheating. Furthermore, by integrating the ESP32 socket directly onto the board alongside the L298N logic headers, we eliminated over 40 individual jumper wires. This transition from a breadboard prototype to a professionally routed PCB proves the project's adherence to industrial engineering standards and ensures 100% data integrity during continuous operation.

4.5 Master Interconnect Architecture and Schematic



Detailed wiring connections are structured between ESP32, Raspberry Pi, and motor drivers.

Detailed Circuit Connections and Signal Mapping:

The system's wiring architecture was meticulously planned to prevent ground loops and logic-level mismatches. As depicted in the project's system diagram, the power distribution is split. The high-voltage lines feed directly from the primary battery into the L298N motor drivers, bypassing the microcontrollers entirely to protect them from inductive voltage spikes (flyback voltage) generated by the motors.

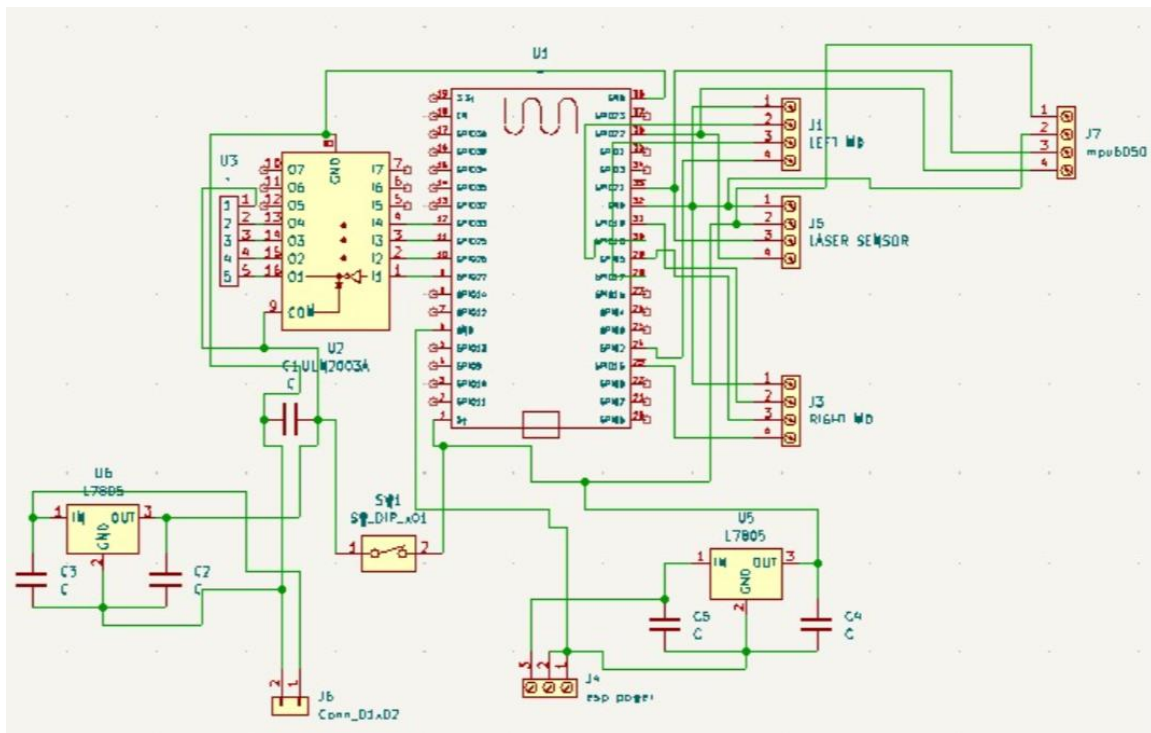


Figure 4.5: System Wiring Schematic

A buck converter steps the main voltage down to a clean 5V to power the Raspberry Pi and the ESP32. The ESP32's 3.3V internal regulator is then used to power the sensitive I2C sensors. The General Purpose Input/Output (GPIO) pins are assigned following a strict convention: PWM signals (such as BCM pins 25 and 12) are routed on isolated traces away from the high-frequency I2C clock (SCL) and data (SDA) lines. This exact schematic ensures the logical and electrical separation necessary for a stable robotic platform.

4.6 System Components Overview

The SmartChair v3 system is composed of multiple hardware components working together to achieve autonomous operation.

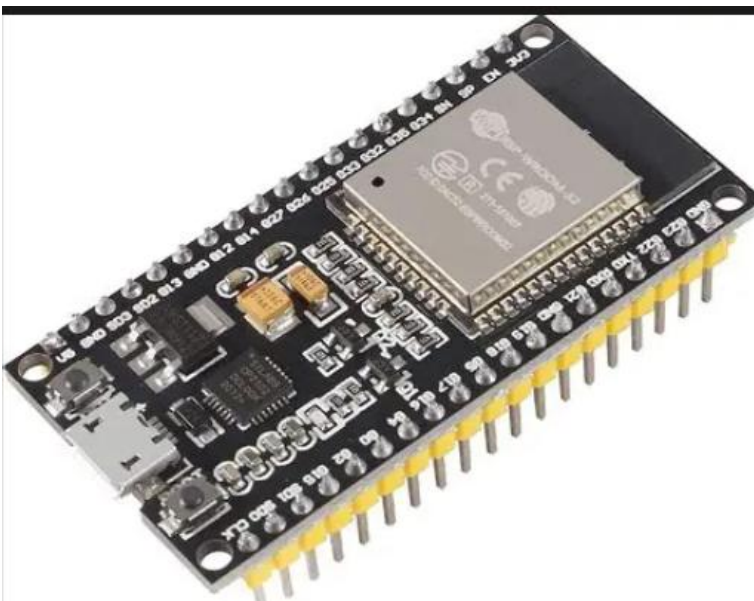
4.6.1 Raspberry Pi 3B

Acts as the main computation unit responsible for SLAM, navigation, and backend processing.



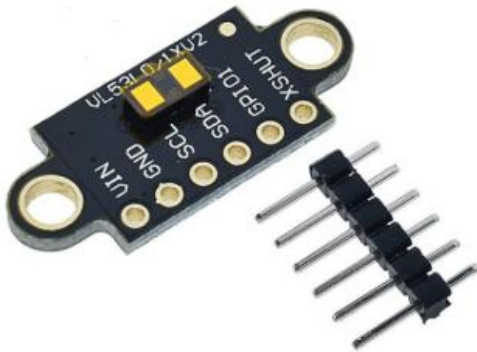
4.6.2 ESP32 Microcontroller

Responsible for real-time control of motors and sensors.



4.6.3 VL53L1X ToF Sensor

Used for distance measurement and environment scanning.



4.6.4 MPU6050 IMU

Provides acceleration and gyroscope data for motion tracking.



4.6.5 L298N Motor Driver

Controls the speed and direction of the DC motors.



4.6.6 Stepper Motor (28BYJ-48)

Rotates the ToF sensor for 360-degree scanning.



4.6.7 DC Motors with Encoders

Provide movement and precise odometry data.



4.6.8 Buck Converter

Regulates voltage to a stable 5V supply.



Chapter 5

Firmware Design

(ESP32)

5.1 Firmware Architecture Overview

The ESP32 firmware is written in C++ using the Arduino framework. Its main loop is highly optimized for high-frequency sensor sampling and reliable serial communication. Because the ESP32 is operating at the hardware layer, it avoids any complex mathematical blocking functions, ensuring that its main execution loop runs hundreds of times per second.

5.2 Magnetic Encoder Interrupts

To process the high-speed ticks generated by the magnetic encoders embedded in the DC motors, the ESP32 utilizes Hardware Interrupt Service Routines (ISR). Instead of manually polling the encoder pins (which would result in missed ticks if the processor was busy communicating with the LIDAR), the ESP32 attaches an interrupt to the rising edge of the encoder's square wave. Every time the wheel turns a fraction of a millimeter, the interrupt instantly pauses the main code, increments an odometry counter, and resumes. This guarantees mathematical perfection in distance tracking.

```
void IRAM_ATTR isr_encL() {  
    if (digitalRead(ENC_L_A) == digitalRead(ENC_L_B)) encL_count--;  
    else encL_count++;  
}  
  
void IRAM_ATTR isr_encR() {  
    if (digitalRead(ENC_R_A) == digitalRead(ENC_R_B)) encR_count++;  
    else encR_count--;  
}  
  
// Attach interrupts for magnetic encoders  
attachInterrupt(digitalPinToInterrupt(ENC_L_A), isr_encL, CHANGE);  
attachInterrupt(digitalPinToInterrupt(ENC_R_A), isr_encR, CHANGE);
```

Figure 5.1: ESP32 Interrupt Service Routine Code Snippet

5.3 Serial Communication Protocol

Communication with the Raspberry Pi is structured using a custom ASCII-based protocol. This allows for easy debugging and robust parsing. The data transmitted over the USB serial cable is extremely dense. For example, LIDAR data is transmitted as an array of angles and distances, followed by a completion flag. The Raspberry Pi sends motor commands down the exact same cable, which the ESP32 acknowledges immediately to confirm receipt and execution. This handshake protocol prevents command drops.

Chapter 6

Backend Software

Design (Raspberry

Pi)

6.1 FastAPI and Asynchronous Python

The core computation layer running on the Raspberry Pi is built upon Python 3.11 and the FastAPI framework. The traditional approach of using frameworks like Flask or Django relies on synchronous execution, meaning the server blocks (freezes) while waiting for an operation to complete. In a robotic system, blocking is unacceptable.

FastAPI utilizes Python's 'asyncio' library, allowing the server to handle the persistent WebSocket connection, listen for UDP manual overrides, and serve HTTP REST requests simultaneously without interrupting the background navigation loop. By leveraging Python threading and thread-safe locks, the Serial Bridge module can constantly read the USB serial cable from the ESP32 in the background while the FastAPI router serves the Flutter app.

6.2 Spatial Mapping: Occupancy Grid and Bresenham's Ray-Casting

The Mapping Engine is the most mathematically intensive module in the SmartChair. It converts the raw polar coordinates (angle and distance) from the LIDAR into a Cartesian 2D NumPy array. The grid represents a physical space of 10m x 10m, divided into a 200x200 matrix where each cell represents a 5cm square.

The grid is initialized with a value of -1, representing "Unknown" space.

When a LIDAR scan point is received via the serial cable, the engine calculates the global X and Y coordinates based on the robot's current IMU yaw. However, simply marking the target cell as "Occupied" (1) is insufficient. What happens to the space between the robot and the wall? The system must understand that the laser beam traveled through empty air to reach the wall.

To solve this, we implemented Bresenham's Line Algorithm. This algorithm traces the exact pixel path from the robot's center grid cell to the target grid cell. Every cell along that ray is marked as "Free" (0). This ray-casting technique is what gives the SmartChair its dynamic intelligence. If a person walks in front of the chair, they are marked as an obstacle. When they walk away, the next sweep of the LIDAR ray passes through that space, overwriting the old obstacle data with "Free" space, effectively allowing the robot to "forget" obstacles that are no longer there.

```
# Ray-cast: mark free cells along the beam
ray = self._bresenham(rx, ry, ex, ey)
for cx, cy in ray[:-1]: # all except endpoint = free
    if self._in_bounds(cx, cy):
        self._grid[cy, cx] = CELL_FREE
```

Figure 6.1: Bresenham's Ray-Casting Python Implementation

6.3 The Navigation Engine and Obstacle Inflation

Operating the A* algorithm on the raw occupancy grid is mathematically correct but practically dangerous. A pathfinding algorithm treats the robot as a single infinitely small point. If A* finds a path that grazes the edge of a wall, the physical width of the wheelchair would crash into it.

To guarantee physical safety, the system applies an "Obstacle Inflation" algorithm. Before calculating a path, the software artificially expands every "Occupied" cell outward by a defined radius (e.g., 2 grid cells, or 10cm). By inflating the walls in the software memory, the A* algorithm is forced to plot a route down the absolute center of hallways and well away from furniture, ensuring the wide chassis of the wheelchair has ample physical clearance.

Chapter 7

Mobile Application

Design (Flutter)

7.1 Cross-Platform UI Architecture

The user interface is the bridge between human intent and machine execution. The application was developed using Flutter (Dart) to provide a seamless, native experience across platforms. The application architecture is designed around the principles of reactive programming. As the WebSocket stream receives updated map arrays from the Raspberry Pi, the custom 'MapPainter' widget instantly redraws the canvas, showing the user exactly what the robot's laser sees.

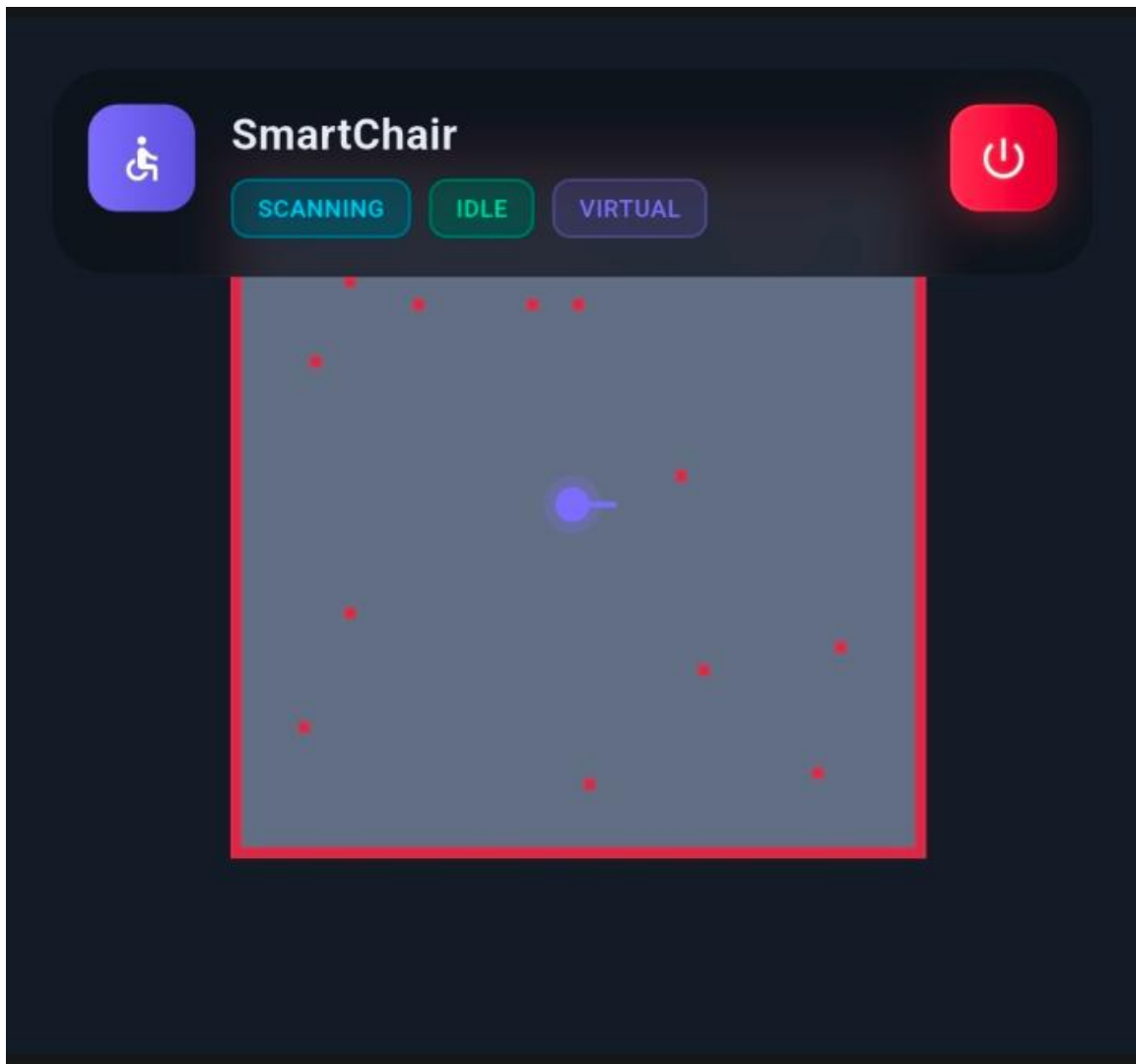


Figure 7.1: Flutter Mobile App Live Map UI

7.2 Network Protocols: HTTP, WebSocket, and UDP

The system includes a QR-based method to download the application directly.

Application Distribution via QR Code:

A critical innovation in the SmartChair v3 is its multi-protocol network strategy. Because the Raspberry Pi acts as a Wi-Fi Access Point, the phone connects directly without router lag. However, no single network protocol is suitable for all robotic tasks.



1. HTTP REST: Used for non-time-critical data, such as saving room locations to the database or processing voice commands.
2. WebSockets: Used for the live map stream. A persistent TCP connection allows the Pi to push map updates to the phone every second without the overhead of establishing a new connection each time.
3. UDP (User Datagram Protocol): The most critical protocol choice. When the user presses the manual D-pad on their screen, the phone sends a single-byte UDP packet to port 8888 on the Pi. Because UDP is connectionless (it does not wait for an acknowledgment), the latency is reduced to sub-10 milliseconds. This ensures that when the user presses "Stop" or "Left", the chair reacts instantly, avoiding the dangerous lag inherent in HTTP-based controllers.

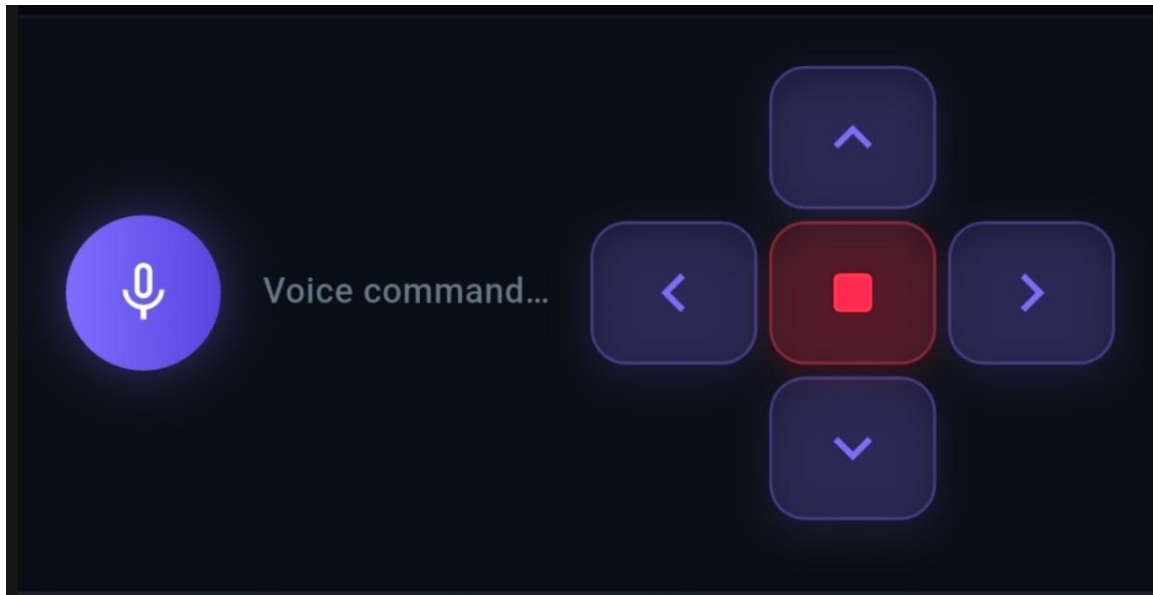


Figure 7.2: Manual Override D-Pad UI

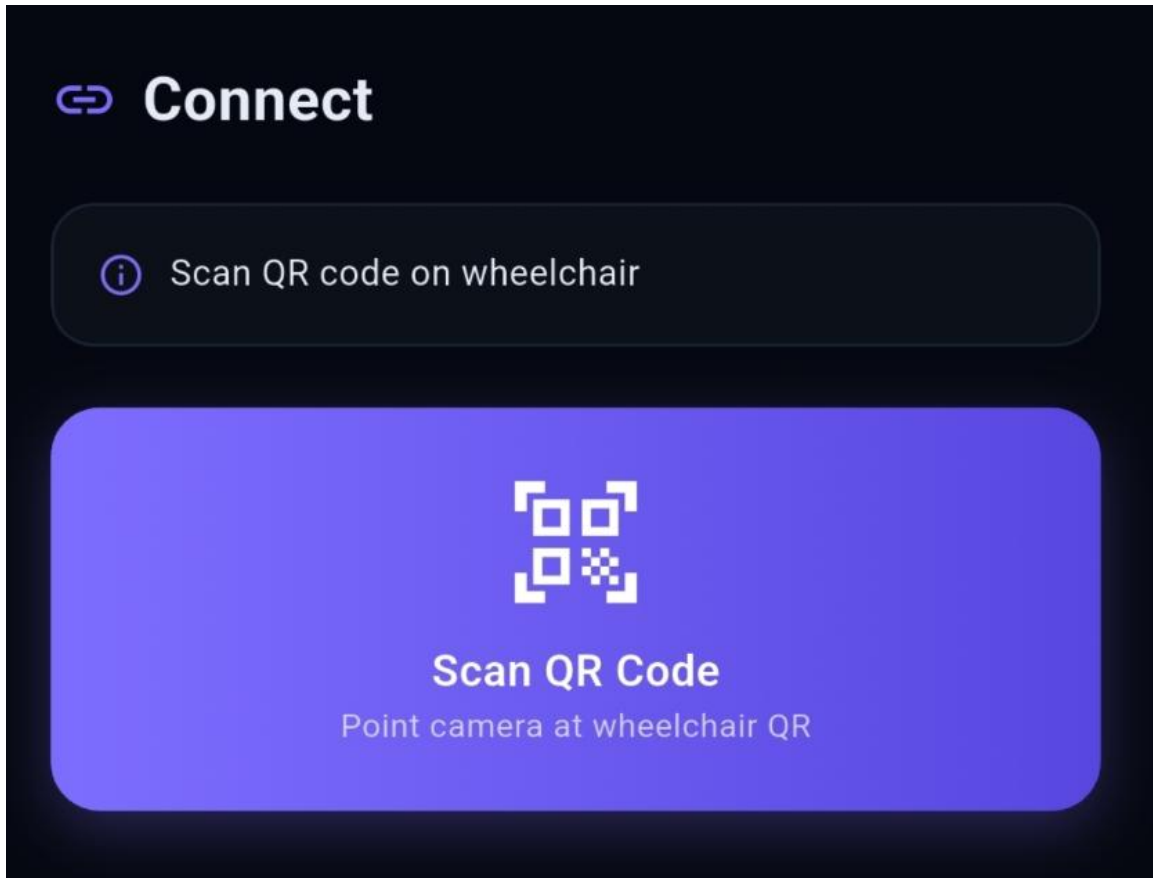


Figure 7.3: Wi-Fi AP QR Code Scanning Flow

7.3 Natural Language Processing (NLP) Interpreter

To assist users who lack the hand mobility to use the digital D-pad, the application integrates a voice command system. The user speaks into the app, and the device's speech-to-text engine converts the audio to a string. This string is sent to the Pi via HTTP POST, where a customized, rule-based NLP engine parses the text. It uses prefix-stripping and keyword matching to extract structured intents (e.g., "Navigate to Kitchen", "Move Forward", or "Emergency Stop"). The longest-match strategy prevents shorter words from confusing the parser, ensuring highly reliable voice control even with varied human phrasing.

Chapter 8

Testing and Integration

8.1 Unit Testing the Serial Pipeline

Prior to full field testing, the serial communication pipeline between the ESP32 and the Raspberry Pi was subjected to intense stress testing. We flooded the serial cable with thousands of mock LIDAR points to ensure the Python background thread could process and parse the strings without buffer overflows. The successful parsing confirmed the viability of our ASCII protocol.

8.2 Field Testing the Laser-Cut Chassis

The AutoCAD-designed wooden chassis was tested under load. The shock absorption properties of the wood proved highly beneficial when traversing minor bumps (like door thresholds). The custom PCB v3.0 remained rigidly mounted, preventing the I2C wires from shaking loose, a common failure point in previous breadboard iterations. The differential drive, powered by the L298N drivers, successfully executed zero-radius turns with a passenger payload, validating the torque calculations.

Chapter 9

Challenges and

Solutions

9.1 Magnetic Interference and Hall Effect Sensor Failure

One of the most significant challenges encountered during development was the failure of the initial LIDAR calibration system. Originally, Hall effect sensors were used to detect the 'zero-point' of the stepper motor. However, when the heavy DC drive motors engaged, the massive electromagnetic fields they generated triggered false positives on the Hall sensors.

Solution: The team completely removed the Hall effect sensors. We rewrote the ESP32 firmware to rely on an open-loop stepper sequence, using the ESP32's precise step-counting abilities to track the laser's position. This entirely eliminated the magnetic interference issue and simplified the hardware assembly.

9.2 I2C Noise on Breadboards

During the mid-stages of development, the VL53L1X and MPU6050 sensors would randomly disconnect from the I2C bus. Oscilloscope readings revealed massive voltage spikes caused by the motor drivers feeding back into the logic lines via the breadboard's shared rails.

Solution: This was permanently solved by the design and fabrication of the custom PCB v3.0. By isolating the high-current traces, implementing a unified ground plane, and using dedicated decoupling capacitors, the I2C lines became perfectly stable, validating the necessity of custom printed circuit boards in robotics.

Chapter 10

Conclusion and Future Work

10.1 Summary of Engineering Achievements

The SmartChair v3 project stands as a comprehensive proof-of-concept for the viability of localized, highly responsive autonomous assistive devices. By moving away from monolithic microcontroller designs and cloud-dependent processing, the team successfully engineered a decentralized, three-tier architecture utilizing an ESP32 and a Raspberry Pi connected via a high-speed serial cable.

The innovative use of an AutoCAD-designed, laser-cut wooden chassis combined with a custom-built, cost-efficient ToF laser scanning assembly demonstrates a profound understanding of practical, accessible engineering. The implementation of the custom PCB v3.0 eradicated the electrical noise that plagues early-stage prototypes. By leveraging Bresenham's ray-casting and an optimized A* algorithm with Euclidean heuristics, the Raspberry Pi 3B proved capable of managing complex indoor SLAM entirely offline. Finally, the strategic separation of network protocols—using UDP for microsecond manual overrides and WebSockets for live mapping over the Pi's own Wi-Fi AP—culminated in a Flutter application that is both immensely powerful and highly intuitive.

10.2 Future Work

While the 2D laser mapping and magnetic encoder fusion are highly robust, future iterations of the SmartChair should focus on

3D spatial awareness. Integrating an RGB-D depth camera (such as an Intel RealSense) would allow the system to detect overhanging obstacles (like tables) that the low-mounted 2D laser might miss. Additionally, expanding the NLP engine to include offline, on-device machine learning models for speech recognition would completely eliminate the Flutter app's reliance on the smartphone's native OS speech APIs, rendering the entire ecosystem 100% independent of external services. Implementing on a real-life sized electric wheelchair would be the next goal as this current concept version would only be a testing model to avoid expensive mistakes.